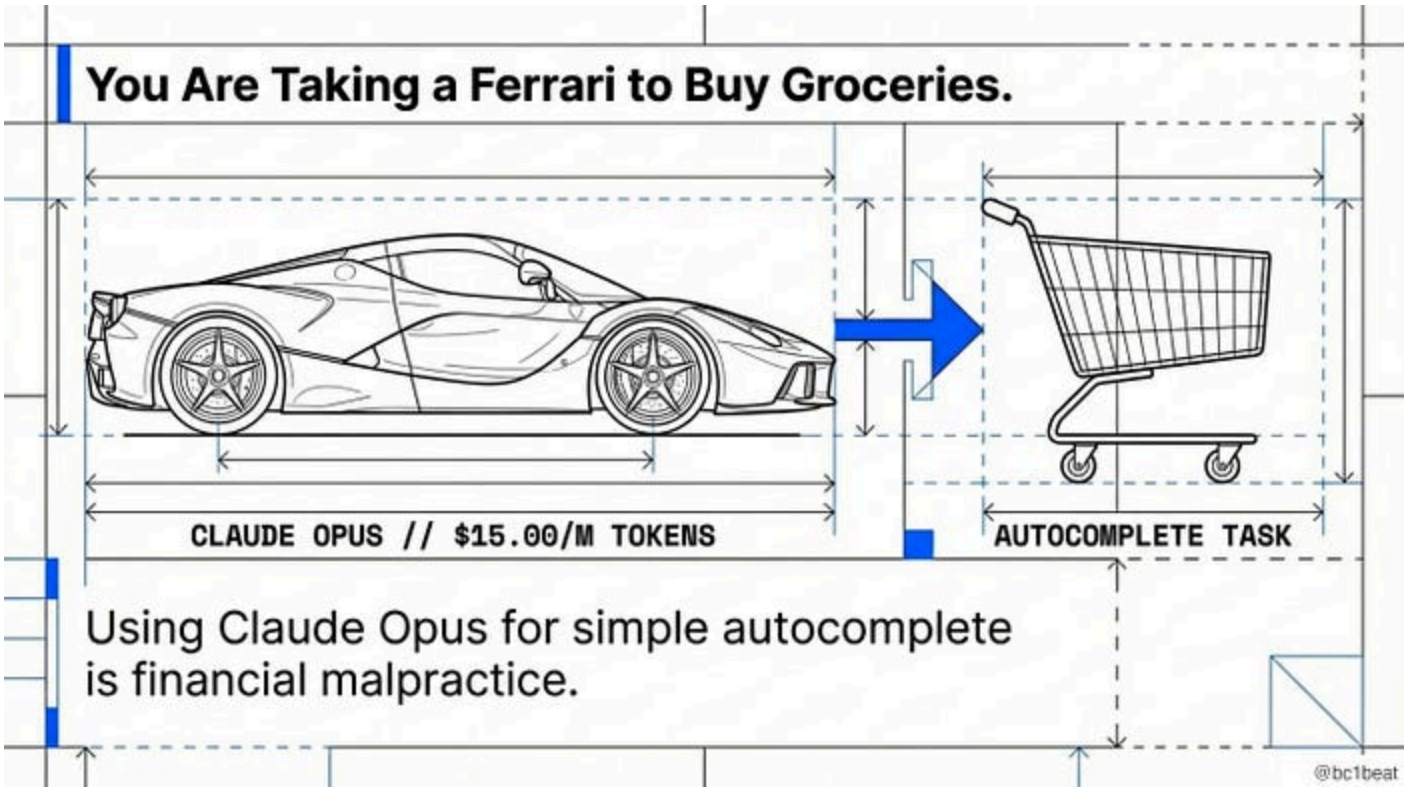
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



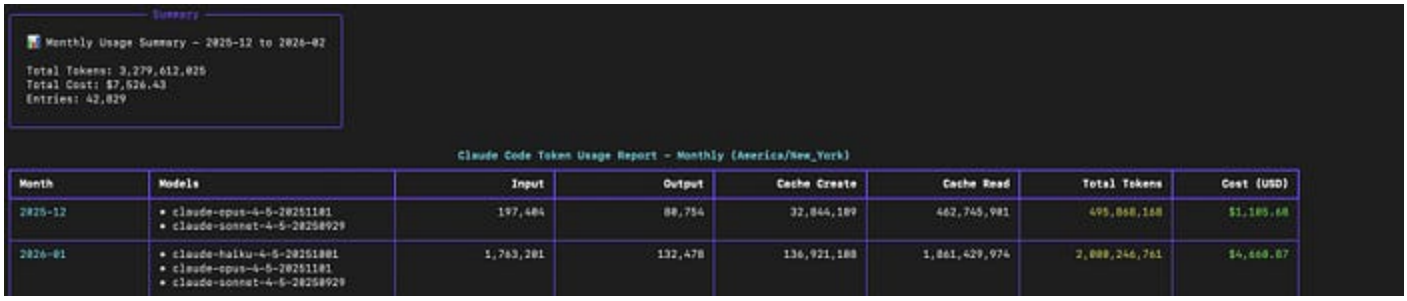
ClawRouter: Anthropic charged me $4,660—How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

Agent Native Follow

~10 min read · February 6, 2026 (Updated: February 6, 2026) · Free: No

I use OpenClaw every day, but I never stopped to think about what I was actually paying for.



Summary

Monthly Usage Summary — 2025-12 to 2026-02

Total Tokens: 3,279,612,025
Total Cost: $7,626.43
Entries: 42,829

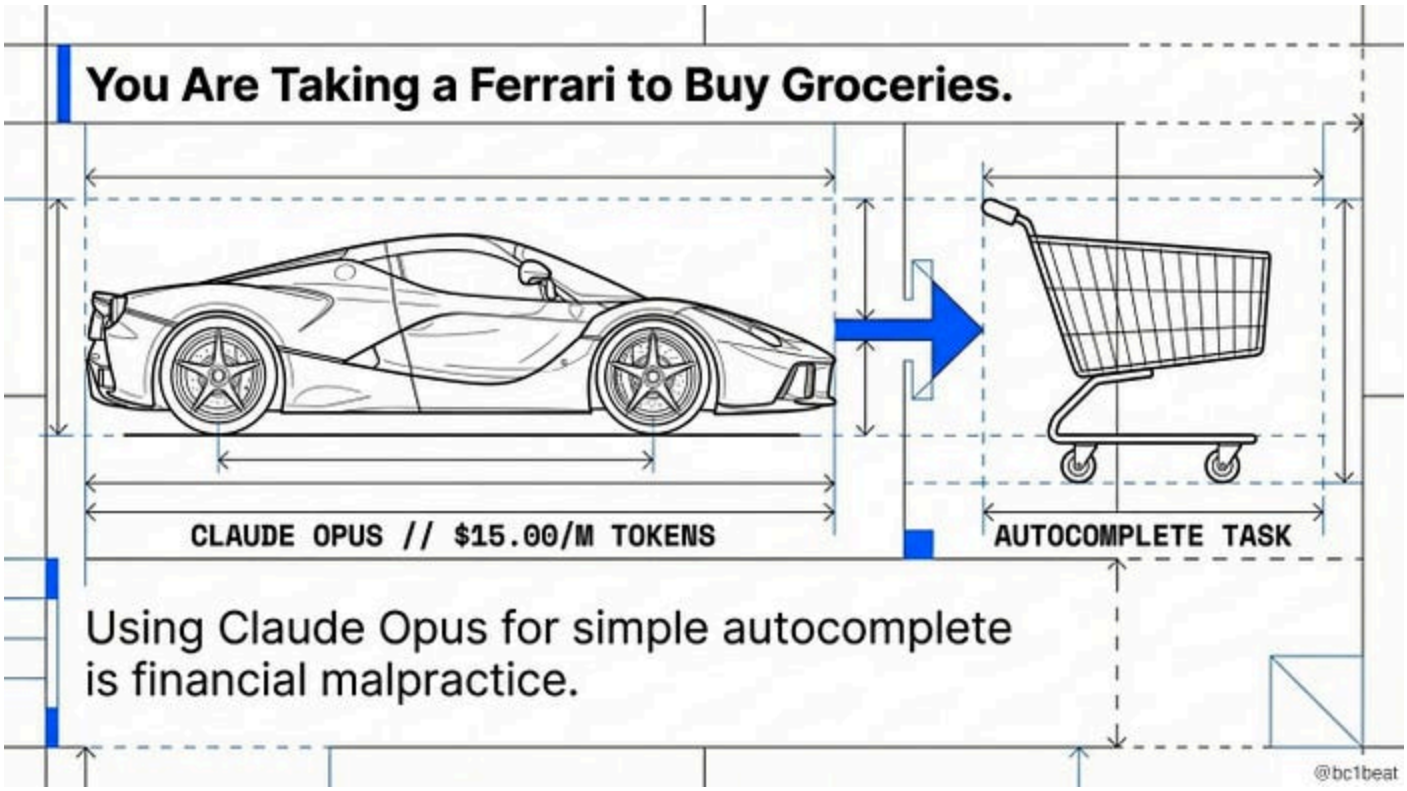
Claude Code Token Usage Report — Monthly (America/New_York)

Month	Models	Input	Output	Cache Create	Cache Read	Total Tokens	Cost (USD)
2025-12	• claude-opus-4-5-20251101 • claude-sonnet-4-5-20250929	197,484	88,754	32,844,189	462,745,981	495,868,168	$1,185.68
2026-01	• claude-haiku-4-5-20251001 • claude-opus-4-5-20251101 • claude-sonnet-4-5-20250929	1,763,201	132,478	136,921,188	1,861,429,974	2,000,246,761	$4,660.07

Most requests, no matter how simple, were routed to the **Claude Opus** model at $75 per million tokens.

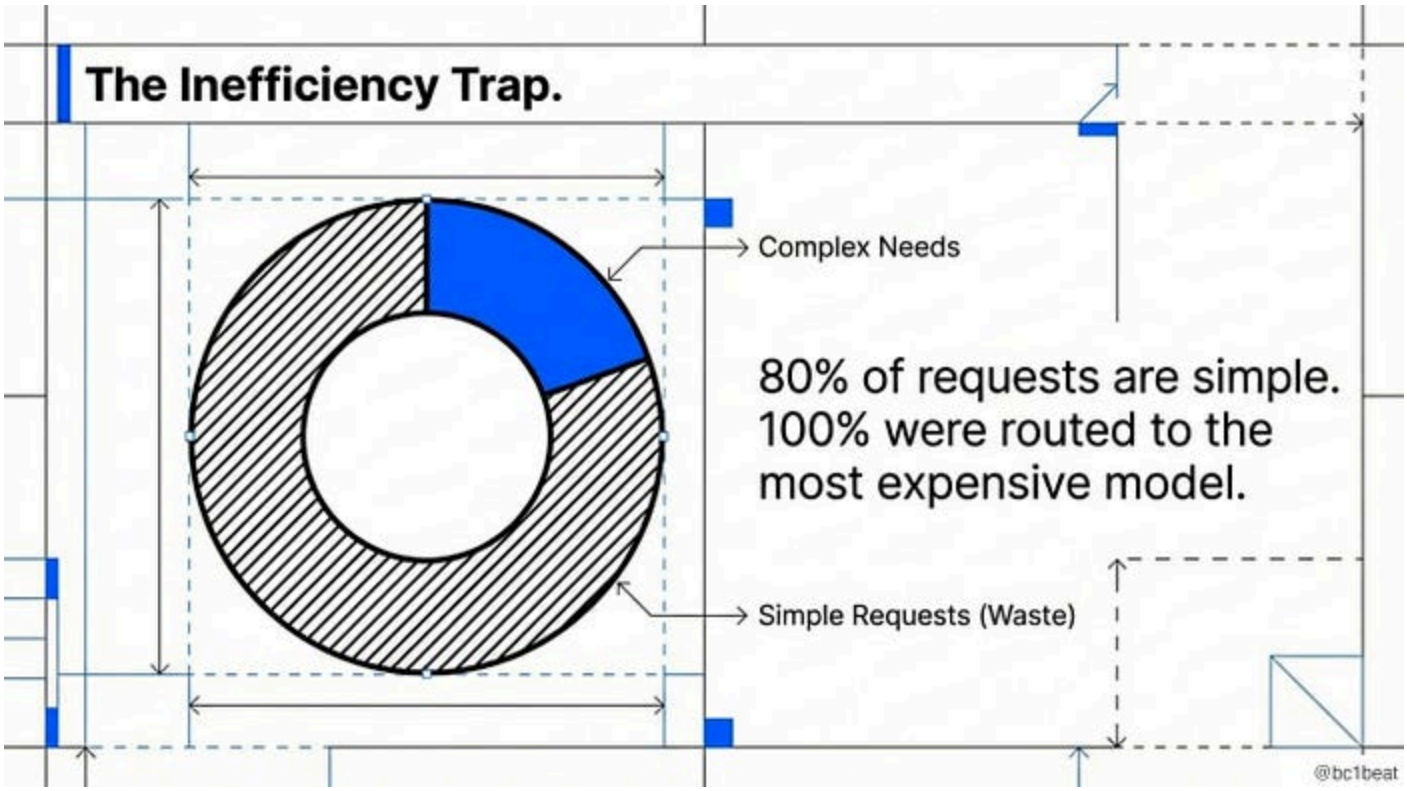
If you don't have a Medium membership, you can read the article here.

It was like taking a Ferrari to the grocery store.



Most of my requests are simple. Autocomplete this line. Explain this error. Fix this syntax. Maybe twenty percent of what I do actually requires a powerful model.

I don't know when requests were routed to the most expensive model. Because switching models manually is annoying, I just left everything on Opus and hemorrhaged money.



This realization kept me up at night. So I built something to fix it.

> This is a guest post written by bc1beat and it deep dives into smart LLM routing for agentic applications. We have no affiliation with ClawRouter, aside from introducing it to developers building multi-agent systems with OpenClaw. Enjoy the read and drop your questions in the comments.

Getting started with ClawRouter

ClawRouter is a smart, local router that sits between OpenClaw (or any OpenAI-compatible client) and the AI providers.

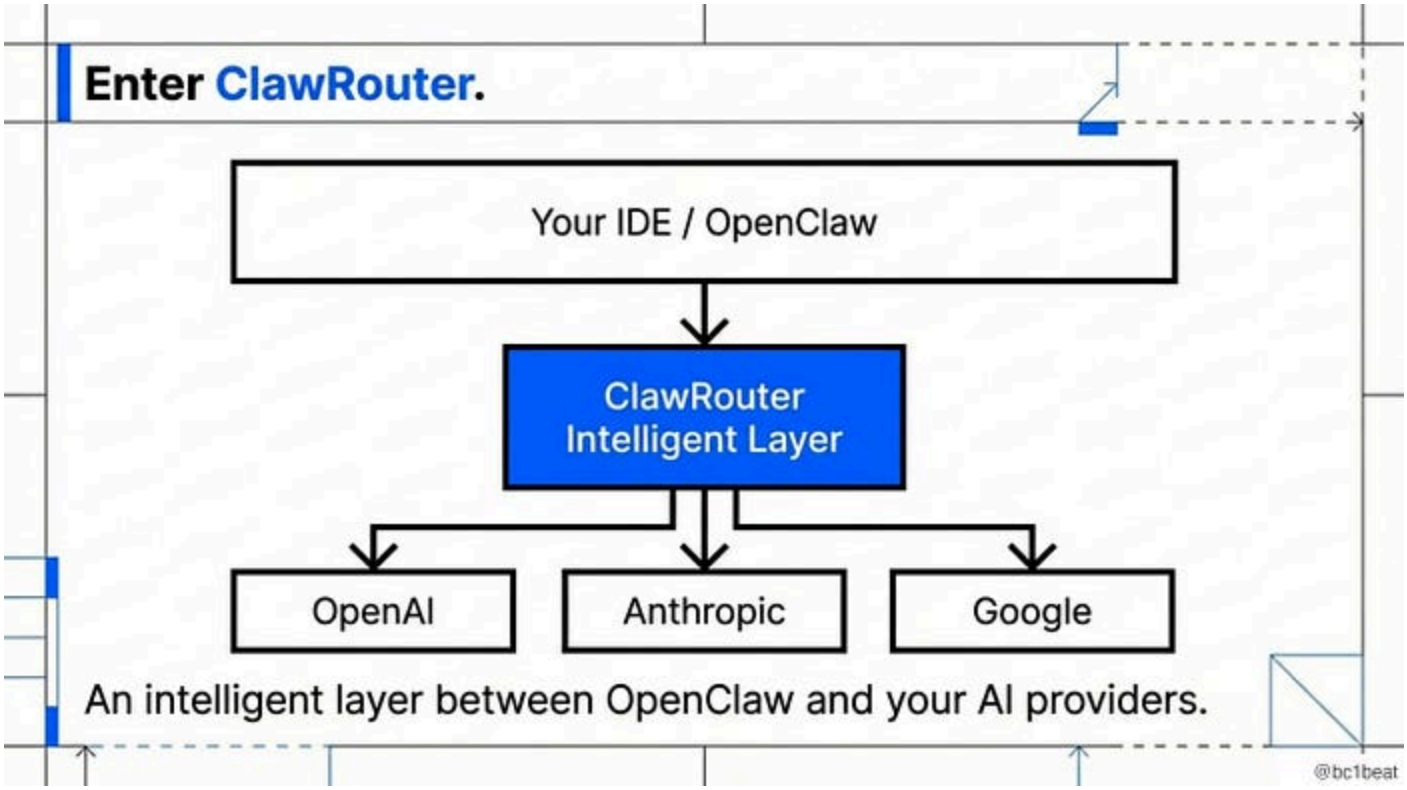
Every time you send a request, it looks at what you're asking and automatically picks the **cheapest model** that can handle it.

A typical workflow looks like this:

Task	Model	Cost per Million Tokens	Why Use It
What is 2+2?	DeepSeek	$0.27/M	saved 99%
Summarize this article	GPT-4o-mini	$0.60/M	saved 99%
Build a React component	Claude Sonnet	$15.00/M	best balance
Prove this theorem	o3	$10.00/M	reasoning
Run 50 parallel searches	Kimi K2.5	$2.40/M	agentic swarm

The routing decision happens locally on your machine.

There are **no API calls** involved and no added latency, the scoring runs in under **1ms**.



To start saving money, install the plugin, fund your wallet and enable the smart model:

```
# 1. Install — auto-generates a wallet on Base
openclaw plugins install @blockrun/clawrouter

# 2. Fund your wallet with USDC on Base (address printed on install)
$5 is enough for thousands of requests

# 3. Restart OpenClaw to load the plugin
openclaw restart
```

Every request now routes through BlockRun with x402 micropayments.

> x402 is a new open payment protocol developed by Coinbase that enables instant, automatic stablecoin payments directly over HTTP.By reviving the HTTP 402 Payment Required status code, x402 lets services monetize APIs and digital content onchain, allowing clients, both human and machine, to programmatically pay for access without accounts, sessions, or complex authentication.

To enable smart routing, add to `~/.openclaw/openclaw.json`:

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "blockrun/auto"
      }
    }
  }
}
```

Or use `/model blockrun/auto` in any conversation to switch on the fly.

Already have a funded wallet?

You can export `BLOCKRUN_WALLET_KEY=0x`… and if you want to use a specific model, you can use `blockrun/openai/gpt-4o` or `blockrun/anthropic/claude-sonnet-4`, and till get x402 payments and usage logging.

Let's see ClawRouter in action:

Now let me verify the wallet was saved: 08:25

Got it! Your ClawRouter wallet address is:

`0xd394Bb2DBdF6d898b3810664968202F41E2f8B5C`

This is a freshly generated wallet on **Base** (L2). To use ClawRouter's 30+ models, you'll need to fund it with **USDC on Base** — send some USDC to that address.

Note: The plugin's `activate()` wasn't being called by OpenClaw automatically (possibly a gateway version thing), so I triggered it manually to generate the wallet. The key is saved securely at `~/.openclaw/blockrun/wallet.key`. 08:25

i have gave you one dollar, help me call grok check @hosseeb opinion about ai agent. 08:32 ✓✓

Let me spawn a Grok-powered sub-agent to research @hosseeb's AI agent takes: 08:32

Spawned a Grok sub-agent to dig into @hosseeb's AI agent takes. It's running on `xai/grok-3` via your ClawRouter — that $1 USDC is at work! 💰

I'll ping you when Grok comes back with the analysis. 08:32

ClawRouter in action

The flow:

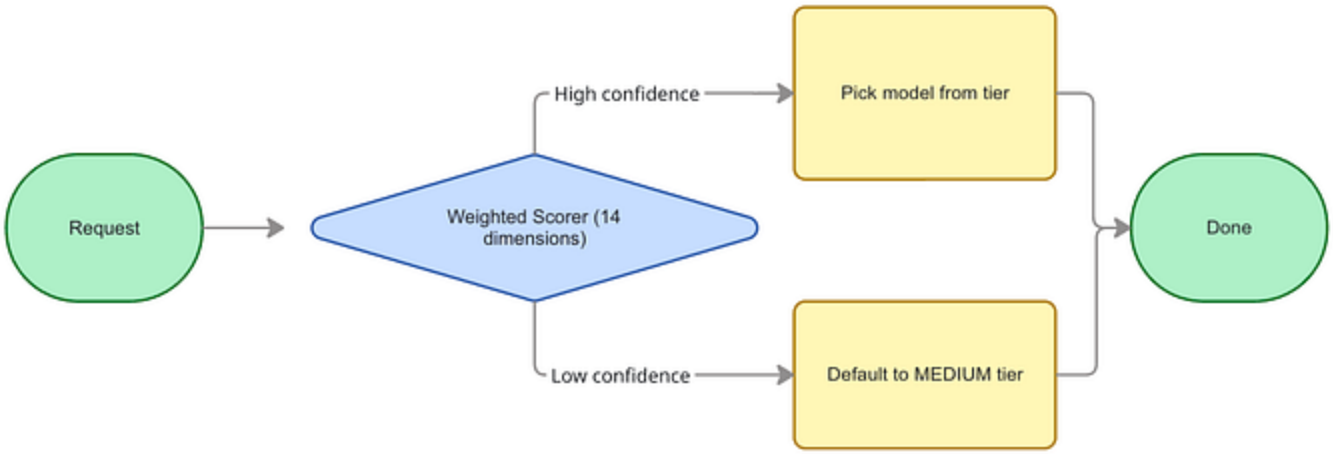
1. Wallet auto-generated on Base (L2) and saved securely at `~/.openclaw/blockrun/wallet.key`
2. Fund with $1 USDC, which is enough for hundreds of requests
3. Request any model, e.g. "help me call Grok to check @hosseeb's opinion on AI agents"
4. ClawRouter routes it, e.g. spawns a Grok sub-agent via `xai/grok-3`, pays per-request

No API keys. No accounts. You can just fund and go.

Architecture: local, open, and agent-friendly

ClawRouter's architecture is beautifully simple:

(1) Weighted Scorer: a rule-based classifier looks at your prompt (and optional system prompt) across **14 dimensions** such as reasoning markers, code presence, simple vs complex indicators, token length, technical terms, imperative verbs, constraints, output format, domain specificity, and more.



Each dimension is weighted, and the weighted sum goes through a sigmoid function to produce a *confidence score* and a *tier* (SIMPLE, MEDIUM, COMPLEX or REASONING).

If two or more reasoning markers are present it jumps straight to the REASONING tier.

Tier	Primary Model	Cost/M	Savings vs Opus
SIMPLE	deepseek-chat	$0.27	99.6%
MEDIUM	gpt-4o-mini	$0.60	99.2%
COMPLEX	claude-sonnet-4	$15.00	80%
REASONING	o3	$10.00	87%

(2) Model Selector: given a tier and confidence, the selector maps to the cheapest capable model. It also computes the estimated cost and the savings compared to a baseline of **Claude Opus**.

(3) x402 Signer: requests to BlockRun's API go through a local proxy that implements the [x402 micropayment protocol](). Payment is authentication: your wallet signs the cost (displayed in the 402 header), and the request continues. There are no API keys, no accounts and no custodial balances; the USDC stays in your wallet until spent.

Because routing and payment happen locally, **privacy** and **latency** are preserved.

The proxy includes clever optimizations: response deduplication (30s cache), streaming heartbeats to prevent upstream timeouts, and payment pre-authorization that skips extra round trips.

How the scoring works

The heart of ClawRouter is the 14-dimension weighted scorer.

Here are a few highlights from `src/router/rules.ts`:

- **Token count:** short prompts (<50 tokens) push the score towards the SIMPLE tier while long prompts (>500 tokens) push towards COMPLEX.
- **Keyword matches:** lists of reasoning words (e.g. *prove*, *theorem*, *step by step*) contribute up to +1.0 to the score; code keywords (*function*, *import*, backticks) contribute up to +1.0; simple indicators (*what is*, *define*) actually subtract, pushing you to the SIMPLE tier.
- **Multi-step patterns** and **question complexity:** multiple steps or several question marks indicate more reasoning.
- **Imperative verbs, constraints, output formats:** these incremental features help differentiate instructions that ask for code generation or structured output.
- **Domain specificity and negation complexity:** detect specialized domains (quantum, genomics) or multiple negations, raising the tier when necessary.

The weighted sum is calibrated via a sigmoid.

If the confidence is below a configurable threshold, the router falls back to the MEDIUM tier (DeepSeek/GPT-4o-mini).

Overriding thresholds or adding custom keywords is as simple as editing your `openclaw.yaml`:

Copy

```yaml
# Override tier models
plugins:
  - id: "@blockrun/clawrouter"
    config:
      routing:
        tiers:
          COMPLEX:
            primary: "openai/gpt-4o"
          SIMPLE:
            primary: "google/gemini-2.5-flash"
# Override scoring weights
routing:
  scoring:
    reasoningKeywords: ["proof", "theorem", "formal verification"]
    codeKeywords: ["function", "class", "async", "await"]
```

Single agent usage: saving money with zero friction

For most developers, the easiest way to adopt ClawRouter is via the OpenClaw plugin:

1. **Installation and wallet creation:** running `openclaw plugins install @blockrun/clawrouter` automatically creates a wallet in `~/.openclaw/blockrun/wallet.key` and prints its address. Fund it with a small amount of USDC on Base and you're ready.
2. **Smart routing:** set `blockrun/auto` as your default model. From this point on, every request you send (via OpenClaw CLI, your local `openai` Python library, or any other client pointed at the ClawRouter proxy) will automatically go to the right model.
3. **Logging:** usage is logged as JSON lines in `~/.openclaw/blockrun/logs/`, so you can audit costs per request.

Here's a minimal TypeScript example without OpenClaw:

```typescript
import { startProxy } from "@blockrun/clawrouter";

const proxy = await startProxy({
  walletKey: process.env.BLOCKRUN_WALLET_KEY!,
  onReady: (port) => console.log(`Proxy on port ${port}`),
  onRouted: (d) => console.log(`${d.model} saved ${(d.savings * 100).toFixed(0)}%`),
});

// Any OpenAI-compatible client works
const res = await fetch(`${proxy.baseUrl}/v1/chat/completions`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "blockrun/auto",
    messages: [{ role: "user", content: "What is 2+2?" }],
  }),
});

console.log(await res.json());
await proxy.close();
```

The proxy inspects the prompt, selects the cheapest model, signs the micropayment, forwards the request and returns the result.

In my own tests, trivial prompts (e.g. *"Summarize this article."*) are routed to **gpt-4o-mini** at $0.60/M tokens, while deep reasoning (e.g. *"Prove that √2 is irrational."*) goes to **o3** or **Claude Sonnet**.

Observability and budgets

ClawRouter exposes a local health endpoint at `/health` that returns wallet status and optionally your USDC balance.

You can hook into `onLowBalance` and `onInsufficientFunds` callbacks to warn or halt operations. The roadmap includes **spend controls** and an **analytics dashboard**.

Multi-agent patterns: orchestrating swarms without breaking the bank

Agentic frameworks (e.g. OpenClaw, CrewAI, AutoGPT) often spawn multiple sub-agents to solve complex tasks.

Without routing, each sub-agent defaults to the most expensive model, quickly burning your budget.

ClawRouter's design makes it ideal for multi-agent workflows:

- **Decentralized budgeting:** each agent can use the same wallet or its own wallet, paying per request via x402. There's no need to share API keys or centralize credits.
- **Tier-based routing:** sub-agents performing simple tasks (search, summarization, code formatting) get routed to cheap models, while specialized agents (reasoning, research) are escalated to higher tiers.
- **Parallel concurrency:** models like **Kimi K2.5** (Moonshot AI) are optimized for agent swarms, supporting up to 100 parallel agents and stable tool chains over hundreds of calls. By combining Kimi with ClawRouter's scoring, you can orchestrate large swarms cheaply and reliably.

Why Not OpenRouter / LiteLLM?

They're built for developers. ClawRouter is built for agents.

Feature	OpenRouter / LiteLLM	ClawRouter
Setup	Human creates account	Agent generates wallet
Auth	API key (shared secret)	Wallet signature (cryptographic)
Payment	Prepaid balance (custodial)	Per-request (non-custodial)
Routing	Proprietary / closed	Open source, client-side

Agents shouldn't need a human to paste API keys. They should generate a wallet, receive funds, and pay per request programmatically.

Example: multi-agent research assistant

Suppose you're building a research assistant that needs to gather background, summarize papers and derive insights.

You can create sub-agents:

1. **Web scraper:** fetches pages, extracts text. Prompts are simple; routed to `deepseek-chat` at $0.27/M.
2. **Summarizer:** condenses long articles; routed to `gpt-4o-mini`.
3. **Synthesizer:** draws connections and formulates insights; may trigger `claude-sonnet-4`.
4. **Fact checker:** uses search and chain-of-thought; may require `o3` or Opus.

Agents interact via messages passed through a coordinator.

Each call to `blockrun/auto` returns the chosen model, cost estimate and savings, which you can log.

Because routing is local, there is no additional latency, crucial for swarms of dozens of agents.

Below is a pseudocode snippet using a hypothetical multi-agent framework:

Copy

```
from my_agents import WebScraper, Summarizer, Synthesizer, FactChecker
from openai import OpenAI

# Configure OpenAI client to point at local ClawRouter proxy
oi = OpenAI(base_url="http://127.0.0.1:8402/v1", api_key="")

# Each agent uses the auto model
scraper = WebScraper(client=oi, model="blockrun/auto")
summarizer = Summarizer(client=oi, model="blockrun/auto")
synthesizer = Synthesizer(client=oi, model="blockrun/auto")
fact_checker = FactChecker(client=oi, model="blockrun/auto")

# Orchestrate tasks
articles = scraper.scrape("topic: agentic AI")
summaries = [summarizer.summarize(a) for a in articles]
draft = synthesizer.synthesize(summaries)
verified = fact_checker.verify(draft)
print(verified)
```

Each call will automatically choose the cheapest model appropriate for the prompt.

You can introspect routing decisions via the callback `onRouted` to see savings and adjust your agent's strategy if certain calls consistently trigger expensive models.

Patterns and insights

- **Chunk your prompts:** smaller prompts often stay in the SIMPLE tier, so break large documents into chunks before summarizing. This helps routing stay cheap.
- **Explicit constraints:** if your agent cares about output format (JSON/YAML), include it in the prompt. The scoring logic detects output format keywords and may route to models better at structured output (e.g. GPT-4o vs DeepSeek).
- **Override when necessary:** for deterministic evaluation or benchmarking, override the selected model by specifying `openai/gpt-4o` or another id in the request. You still benefit from x402 and logging.
- **Monitor balances:** use `onLowBalance` and `onInsufficientFunds` to gracefully degrade your agent's capabilities when funds are low: skip optional tasks or route everything to a lower tier.
- **Use fallback chains:** ClawRouter supports cascade routing (planned). For now, you can manually implement fallback: call `blockrun/auto`, check if the response quality is sufficient, and if not, re-issue the prompt with `claude-sonnet-4`.

Beyond the router: ecosystem and roadmap

ClawRouter is part of **BlockRun's** pay-per-request AI infrastructure.

Together with OpenClaw, it aims to make agents autonomous in both reasoning and billing.

The roadmap includes cascade routing, spend controls and an analytics dashboard.

For developers building agentic applications, this translates to:

- Fine-grained cost controls and budgets per agent or project.
- Unified access to 30+ models across OpenAI, Anthropic, Google, DeepSeek, xAI, and Moonshot.
- Transparent, open-source routing logic that you can extend or embed in your own tools.

Thoughts

The router's local-first philosophy means privacy, speed and flexibility are in your hands.

If you build agents or any application that invokes LLMs, give ClawRouter a try.

You might be surprised at how much efficiency you can unlock when the right tool is picked for every job.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs "Context"

When I'm building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

Open Source Perplexity: Perplexica is Self-Hosted AI Search With Citations

Perplexica is an open-source, AI-powered search engine inspired by Perplexity-style UX.

medium.com

Qwen3-Max-Thinking Outperforms Claude Opus 4.5 and Gpt-5.2: The Panic Is Real

Alibaba’s Qwen3 family of models has become a major alternative in the LLM ecosystem.

medium.com
#openclaw#clawdbot#openclaw-bot#agent-routing#claude-opus

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel